Convolutional Neural Networks

Week 13 — Session 1: From hand-designed kernels to learned filters

Practical Image Processing

Lecture 13.1 ▪ 60–70 min

The big idea

- Week 3: *we* designed kernels (blur, Sobel) by hand.
- Week 12: hand-designed features (HOG) hit a ceiling.
- This week: let the network **learn the kernels** from data — and learn the classifier at the same time.
- **Today (Session 1):** what a convolutional layer *is*, and how the pieces fit into a CNN.

Where we are heading

A small CNN that reads digits (0–9) at **97%** — the DocScan meter-digit reader, trained from scratch. Session 2 trains it; today we build the machine.

Roadmap — Session 1

From kernels to learned filters

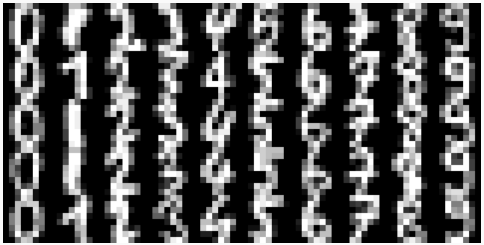
Anatomy of a CNN

In-class exercise

Wrap-up

From kernels to learned filters

Our running problem: reading meter digits



- Each sample is a tiny 8×8 grayscale digit.
- DocScan must turn the water-meter crop into the number it shows.
- Same task all week: input 8×8 image $\rightarrow$ one of 10 classes.

Key idea

Keep this concrete task in mind — every abstract idea today plugs straight into it.

Convolution: you already know it

- Week 3: slide a small kernel over the image, multiply-and-sum $\rightarrow$ a filtered image.
- A Sobel kernel detects edges; a blur kernel smooths. *We* chose the numbers.
- A **convolutional layer** is the same operation — but the kernel numbers are **learned**.

Key idea

A CNN is "convolution with kernels chosen by training instead of by hand."

A convolution, by hand

Slide a 3×3 kernel over the highlighted patch and multiply-and-sum.

Input patch

2	0	1
4	1	0
2,3,1		

Kernel (Sobel-x)

1	0	-1
2	0	-2
1,0,-1		

Multiply–sum

$$2(1) + 0 + 1(-1) + 4(2) + 0 + 0(-2) + 2(1) + 0 + 1(-1) = 2 - 1 + 8 + 2 - 1 = \mathbf{10}$$

One output pixel $= 10$. Repeat over every position to get the whole feature map.

Padding and stride

- **Padding**: add a border of zeros so the kernel can sit on edge pixels — output stays the same size.
- **Stride**: how far the kernel hops each step. Stride 2 halves the output.

Math

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Our digit conv

$H = 8,\ K = 3,\ P = 1,\ S = 1$:

$$\frac{8+2-3}{1} + 1 = 8$$

so an 8×8 input stays 8×8. That "same" padding is what we use.

What the network learns



- These are the 8 first-layer 3×3 filters our CNN *learned* for digits.
- Nobody designed them — gradient descent found edge/stroke detectors that help read digits.
- Early layers learn edges; deeper layers combine them into parts, then whole shapes.

Feature maps

- Applying one learned filter to the image gives a **feature map**: where that pattern appears.
- A layer has many filters $\rightarrow$ many feature maps (a "stack" of pattern detectors).
- The next layer convolves over that stack, combining simple patterns into complex ones.

Math

Conv layer: input stack $\rightarrow$ (learned filters) $\rightarrow$ output stack of feature maps.

From one image to a stack of maps



- Each of the 8 learned filters produces its own 8×8 map.
- Together they form a **tensor**: 8 channels of "where this pattern is."
- Depth = number of filters; height/width = spatial size after padding/stride.

Anatomy of a CNN

The building blocks

Layer	Does	Why
Convolution	learned filters $\rightarrow$ feature maps	detect patterns
ReLU	$\max(0, x)$, elementwise	add non-linearity
Pooling	downsample (e.g. 2×2 max)	shrink, add invariance
Fully-connected	flatten $\rightarrow$ class scores	make the decision
Softmax	scores $\rightarrow$ probabilities	pick the class

Key idea

Conv+ReLU+Pool blocks stack up, ending in a classifier — the standard CNN recipe.

The convolution layer, counted

A conv layer is defined by: #input channels C_{in}, #filters C_{out}, kernel $K \times K$.

Weights in a conv layer

params $= C_{\text{out}}\big(C_{\text{in}} \cdot K \cdot K\big) + C_{\text{out}}$ (the $+C_{\text{out}}$ is one bias per filter)

- Our first layer: $C_{\text{in}}{=}1$, $C_{\text{out}}{=}8$, $K{=}3 \Rightarrow 8(1 \cdot 9) + 8 = \mathbf{80}$ weights.
- Crucial: the same 80 numbers are reused at *every* pixel — **weight sharing**.

Good practice

Weight sharing is why CNNs need far fewer parameters than a fully-connected net on pixels.

Why ReLU (non-linearity)?

- Stacking pure convolutions is still just one big linear filter — no extra power.
- **ReLU** ($\max(0, x)$) inserts a non-linearity between layers.
- That lets deep stacks represent complex, curved decision boundaries.

Math

Without a non-linearity, ten conv layers collapse into one. ReLU is what makes "deep" meaningful.

ReLU, by hand

ReLU clips every negative value to zero and leaves positives untouched.

Feature map (raw)

-3	5	-1
2	-4	0
7,-2,3		

$\rightarrow$

After ReLU

0	5	0
2	0	0
7,0,3		

- Negatives $\rightarrow$ 0; positives pass through. Cheap and fast.
- Keeps only the "this pattern is present" evidence, discards the rest.

Why pooling?

- **Max-pooling** keeps the strongest response in each small window, halving the size.
- Gives **translation tolerance**: a digit shifted a pixel still activates the same units.
- Shrinks computation and grows the **receptive field** of later layers.

✔ Good practice

Pooling = "I saw this pattern somewhere in this region," not "exactly at this pixel."

Max-pooling, by hand

Split into 2×2 windows; keep the max of each.

4×4 feature map

1	3	2	0
4	2	1	5
0	1	6	2
2,3,1,0			

$\rightarrow$

2×2 pooled

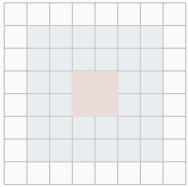
4	5
3,6	

top-left window $\max(1, 3, 4, 2) = 4$

Four numbers become one; the map shrinks $4 \times 4 \rightarrow 2 \times 2$, keeping the strongest signals.

Receptive field: seeing more with depth

one deep unit “sees” a wide input region



- A unit after one 3×3 conv sees a 3×3 patch (coral).
- After a pool + another conv it sees a much larger region (blue).
- Deep units integrate whole-digit context, even though every kernel is tiny.

Fully-connected + softmax: the decision

- **Flatten**: unroll the final map stack into one long vector.
- **Fully-connected** layer: every input times a weight $\rightarrow$ 10 raw scores (**logits**).
- **Softmax**: turn the 10 logits into probabilities that sum to 1.

Math

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$ — biggest logit becomes the most probable class.

Our digit CNN, end to end

- Input 8×8 digit $\rightarrow$ Conv(8 filters)+ReLU+Pool $\rightarrow 4 \times 4$.
- $\rightarrow$ Conv(16 filters)+ReLU+Pool $\rightarrow 2 \times 2 \rightarrow$ flatten (64) $\rightarrow$ FC $\rightarrow$ 10 scores.
- Softmax turns the 10 scores into probabilities; the largest is the predicted digit.

Key idea

Two conv blocks and one linear layer — tiny by modern standards, yet it reads digits at 97%.

Shape bookkeeping (track the tensor)

Stage	**Output shape** $(C \times H \times W)$	**Size change**
Input	$1 \times 8 \times 8$	—
Conv1 (8, pad 1)	$8 \times 8 \times 8$	depth $1 \to 8$
Pool (2×2)	$8 \times 4 \times 4$	spatial halved
Conv2 (16, pad 1)	$16 \times 4 \times 4$	depth $8 \to 16$
Pool (2×2)	$16 \times 2 \times 2$	spatial halved
Flatten	64 vector	$16 \cdot 2 \cdot 2$
FC	10 scores	one per digit

Key idea

Spatial size shrinks, depth grows: the net trades "where" for "what."

Parameter budget of the digit CNN

Layer	Formula	Params
Conv1 ($1 \to 8,\ 3\times3$)	$8(1 \cdot 9) + 8$	80
Conv2 ($8 \to 16,\ 3\times3$)	$16(8 \cdot 9) + 16$	1,168
FC ($64 \to 10$)	$64 \cdot 10 + 10$	650
Total		**1,898**

- Under 2,000 numbers — a fully-connected net on 8×8 pixels to 10 classes would already have 650.
- Weight sharing keeps the conv layers astonishingly cheap.

In-class exercise

In-class exercise (10 min)

Reason about the machine you just built

1. How is a convolutional layer related to the Sobel kernel from Week 3?
2. Convolve the kernel $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ over the patch $\left(\begin{smallmatrix} 3 & 5 \\ 2 & 4 \end{smallmatrix}\right)$ at one position — what is the output?
3. A 16×16 input goes through Conv(K=3, P=1, S=1) then 2×2 pool. What spatial size comes out?
4. Why won't stacking convolutions *without* ReLU give a more powerful model?

Debrief

- (1) Both convolve a small kernel over the image; Sobel's numbers are fixed, the CNN's are **learned**.
- (2) $0 \cdot 3 + 1 \cdot 5 + 1 \cdot 2 + 0 \cdot 4 = 7$.
- (3) Conv with same padding keeps 16×16; the 2×2 pool halves it to 8×8.
- (4) Composing linear operations is still linear — **ReLU**'s non-linearity is what adds power.

Key idea

You can now trace an image through every layer and predict its shape — the core CNN skill.

Wrap-up

Common pitfalls when reasoning about CNNs

✗ Watch out

- Forgetting the bias term when counting parameters (one per filter, one per output unit).
- Confusing *depth* (number of filters) with *spatial* size (height/width).
- Assuming pooling "loses everything" — it keeps the strongest evidence per region.
- Expecting more layers to help *without* a non-linearity between them.

Good habits

✔ Good practice

- Sketch the tensor shape after *every* layer before you write code.
- Prefer "same" padding ($P = \lfloor K/2 \rfloor$) when you want to control size only via pooling.
- Think of a conv layer as a bank of tiny pattern detectors reused everywhere.

What you can already do

- Explain a convolutional layer as convolution with *learned* filters + weight sharing.
- Compute an output pixel, an output size, a ReLU, and a max-pool by hand.
- Trace a tensor's shape and count parameters through a whole small CNN.
- Name every building block — conv, ReLU, pool, FC, softmax — and say why it is there.

Next session

Session 2 — making the filters good

- How the net *learns* those filters: forward pass, loss, backprop, gradient descent.
- Cross-entropy and softmax with real numbers; learning rate, batches, epochs.
- Reading training curves; regularization; when CNNs beat classical ML.
- We train the digit reader to 97% and ship it as the week's deliverable.

Come with today's architecture firmly in your head — next time we bring it to life.

A CNN is convolution with filters chosen by training.

Today we built the machine: conv, ReLU, pooling, and a classifier. Next: we teach it.